

So sánh Transformer Seq2Seq huấn luyện từ đầu và ViT5 cho tóm tắt trừu tượng văn bản tiếng Việt

Đặng Minh Nhật (24022423), Đặng Duy Anh (24021358),

Lưu Xuân Tân (24022447), Trần Đức Thịnh (24022459)

Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội

Học phần Xử lý ngôn ngữ tự nhiên (2526H_INT3406_80)

Hà Nội, Việt Nam

Ngày trình bày: 22/08/2026

TÓM TẮT

Bài báo nghiên cứu bài toán tóm tắt trừu tượng đơn văn bản cho tin tức tiếng Việt và xây dựng một quy trình đánh giá có thể kiểm tra lại. Bốn hệ thống được so sánh trên cùng tập test gồm 2.000 bài báo: Lead-1, Lead-3, Transformer encoder-decoder huấn luyện từ đầu và checkpoint ViT5 đã huấn luyện trước. Dữ liệu được chuẩn hóa, lọc nhiễu và kiểm soát trùng lặp trước khi chia tập. Cấu hình giải mã được lựa chọn trên validation rồi đóng băng trước khi đánh giá test. Ngoài ROUGE, nghiên cứu sử dụng compression ratio, kiểm tra chất lượng reference và đánh giá source-grounded bằng mô hình ngôn ngữ lớn. Scratch Transformer đạt ROUGE cao nhất với R-1/R-2/R-L lần lượt là 33,84/17,84/28,30. Lead-3 đạt điểm overall cao nhất theo LLM judge nhưng có đầu ra dài hơn đáng kể. ViT5 trôi chảy và súc tích hơn Scratch Transformer, trong khi khác biệt overall giữa hai mô hình sinh chưa rõ ràng theo bootstrap 95% CI. Kết quả cho thấy không tồn tại một hệ thống tốt

nhất theo mọi tiêu chí và chất lượng tóm tắt cần được diễn giải đồng thời theo độ tương đồng reference, độ nén, độ bao phủ và tính đúng sự thật.

Từ khóa: tóm tắt văn bản tiếng Việt, Transformer, ViT5, ROUGE, LLM-as-a-Judge, factuality

I. GIỚI THIỆU

Khối lượng tin tức tiếng Việt tăng nhanh làm nảy sinh nhu cầu rút gọn bài báo thành nội dung ngắn, chính xác và dễ đọc. Tóm tắt trích xuất giữ nguyên câu từ nguồn nên thường có độ trung thực cao, nhưng phụ thuộc mạnh vào cấu trúc bài viết. Tóm tắt sinh có thể diễn đạt linh hoạt và súc tích hơn, song dễ tạo thông tin không có trong bài gốc. Vì vậy, một thực nghiệm đáng tin cậy cần đánh giá đồng thời độ tương đồng với reference, độ nén, tính đúng sự thật và khả năng bao phủ nội dung chính.

Nghiên cứu trả lời bốn câu hỏi: (i) Transformer huấn luyện từ đầu và ViT5 so sánh thế nào với Lead-1/Lead-3 theo ROUGE; (ii) kết luận thay đổi ra sao khi chấm trực tiếp theo source; (iii) compression ratio giải thích trade-off giữa coverage và concise-

ness như thế nào; và (iv) chất lượng reference ảnh hưởng ra sao đến cách diễn giải ROUGE.

Các đóng góp chính gồm: chuẩn hóa manifest theo cùng schema và kiểm soát quan hệ giữa các split; so sánh bốn hệ thống trên cùng ID và cùng evaluator; đóng gói Scratch Transformer cùng tokenizer và cấu hình suy luận; bổ sung reference audit và LLM judge theo thiết kế paired, blind, cân bằng vị trí; và phân tích mối quan hệ giữa ROUGE, factuality, coverage, fluency, conciseness và độ dài đầu ra.

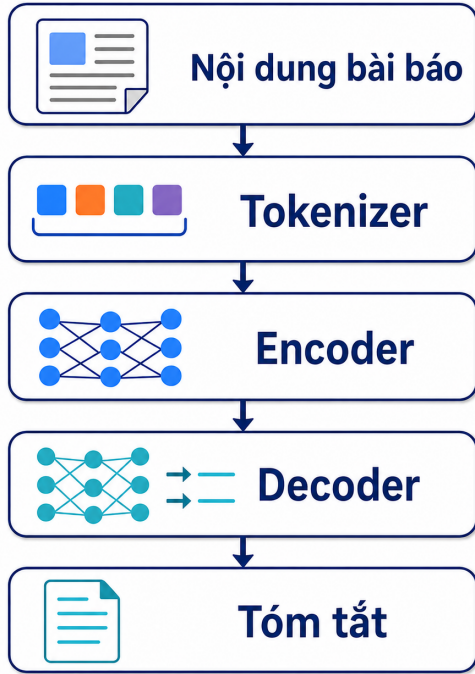


Fig. 1. Pipeline tổng quát của bài toán tóm tắt văn bản tiếng Việt.

II. CƠ SỞ PHƯƠNG PHÁP

A. Tóm tắt trích xuất và tóm tắt sinh

Lead-1 và Lead-3 lần lượt lấy câu đầu hoặc ba câu đầu của bài báo. Hai baseline tận dụng *lead bias*, tức thông tin quan trọng của tin tức thường xuất hiện sớm. Scratch Transformer và ViT5 là các hệ

thống sinh, tạo chuỗi đích theo phân phối xác suất có điều kiện

$$P(\mathbf{y} | \mathbf{x}) = \prod_{t=1}^T P(y_t | y_{<t}, \mathbf{x}). \quad (1)$$

Cách tiếp cận sinh linh hoạt hơn nhưng có thể hallucinate, sai thực thể, sai số liệu hoặc thay đổi quan hệ giữa sự kiện.

B. Attention và kiến trúc Transformer

Transformer mô hình hóa quan hệ giữa mọi cặp token bằng attention thay vì xử lý tuần tự bằng RNN. Với chuỗi nguồn $\mathbf{x} = (x_1, \dots, x_S)$ và chuỗi đích $\mathbf{y} = (y_1, \dots, y_T)$, encoder biến toàn bộ chuỗi nguồn thành memory

$$\mathbf{H}^{(L_e)} = \text{Encoder}(\mathbf{x}) \in \mathbb{R}^{S \times d_{\text{model}}}, \quad (2)$$

còn decoder kết hợp tiền tố $y_{<t}$ với memory để ước lượng phân phối của token kế tiếp. Do đó, cùng một kiến trúc thực hiện hai nhiệm vụ khác nhau: encoder tạo biểu diễn ngữ cảnh hai chiều của bài báo; decoder sinh tóm tắt từ trái sang phải và không được nhìn trước token tương lai.

Embedding và thông tin vị trí. Vì attention tự thân không biết thứ tự token, biểu diễn đầu vào tại vị trí p được tạo bằng tổng của token embedding và positional encoding:

$$\mathbf{h}_p^{(0)} = \sqrt{d_{\text{model}}} E[x_p] + \mathbf{PE}_p. \quad (3)$$

Thừa số $\sqrt{d_{\text{model}}}$ đưa độ lớn của embedding về cùng thang với positional encoding. Mô hình sử dụng

positional encoding hình sin, không có tham số học:

$$\text{PE}_{p,2i} = \sin(p/10000^{2i/d_{\text{model}}}), \quad (4)$$

$$\text{PE}_{p,2i+1} = \cos(p/10000^{2i/d_{\text{model}}}). \quad (5)$$

Các tần số khác nhau giúp mô hình biểu diễn khoảng cách tương đối giữa vị trí, đồng thời tránh phải học một bảng vị trí riêng cho từng độ dài.

Scaled dot-product attention. Từ đầu vào $X \in \mathbb{R}^{L \times d_{\text{model}}}$, mỗi head tạo truy vấn, khóa và giá trị bằng các phép chiếu tuyến tính:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V. \quad (6)$$

Với mask cộng M , attention được tính bởi

$$S_{\text{attn}} = \frac{QK^\top}{\sqrt{d_k}} + M, \quad (7)$$

$$A = \text{softmax}(S_{\text{attn}}), \quad (8)$$

$$\text{Attn}(Q, K, V; M) = AV. \quad (9)$$

Phần tử A_{ij} là mức đóng góp của token j khi cập nhật biểu diễn token i . Phép chia $\sqrt{d_k}$ ngăn tích vô hướng tăng theo số chiều, nhờ đó softmax ít rơi vào vùng bão hòa và gradient ổn định hơn. Mask nhận giá trị 0 tại vị trí hợp lệ và $-\infty$ tại vị trí bị cấm; sau softmax, xác suất của vị trí bị cấm bằng 0.

Multi-head attention. Một attention duy nhất có thể trộn nhiều kiểu quan hệ trong cùng không

gian. Multi-head attention tách không gian biểu diễn thành h không gian con:

$$Q_r = XW_r^Q, \quad K_r = XW_r^K, \quad (10)$$

$$V_r = XW_r^V, \quad (11)$$

$$\text{head}_r = \text{Attn}(Q_r, K_r, V_r; M), \quad (12)$$

$$C = [\text{head}_1; \dots; \text{head}_h], \quad (13)$$

$$\text{MHA}(X; M) = CW^O. \quad (14)$$

Trong Scratch Transformer, $d_{\text{model}} = 384$ và $h = 6$, nên $d_k = d_v = 384/6 = 64$ và hệ số chuẩn hóa là $\sqrt{64} = 8$. Sau khi nối sáu head, tensor trở lại 384 chiều trước phép chiếu W^O . Các head có thể học các quan hệ bổ sung nhau, chẳng hạn đồng tham chiếu thực thể, quan hệ chủ thể–hành động, mốc thời gian và mức độ nổi bật của câu.

Khối encoder Pre-Norm. Với đầu vào lớp $\mathbf{H}^{(\ell-1)}$, một lớp encoder thực hiện

$$\mathbf{A}^{(\ell)} = \text{MHA}(\text{LN}(\mathbf{H}^{(\ell-1)}); M_{\text{src}}), \quad (15)$$

$$\tilde{\mathbf{H}}^{(\ell)} = \mathbf{H}^{(\ell-1)} + \text{Dropout}(\mathbf{A}^{(\ell)}), \quad (16)$$

$$\mathbf{G}^{(\ell)} = \text{FFN}(\text{LN}(\tilde{\mathbf{H}}^{(\ell)})), \quad (17)$$

$$\mathbf{H}^{(\ell)} = \tilde{\mathbf{H}}^{(\ell)} + \text{Dropout}(\mathbf{G}^{(\ell)}). \quad (18)$$

Layer normalization được đặt trước từng sublayer, vì vậy đây là Pre-Norm. Với vector $u \in \mathbb{R}^d$,

$$\text{LN}(u) = \gamma \odot \frac{u - \mu(u)}{\sqrt{\sigma^2(u) + \epsilon}} + \beta. \quad (19)$$

Residual connection cung cấp một đường truyền gần đồng nhất cho cả activation và gradient; Layer-Norm kiểm soát thang đo trước khi đi qua attention hoặc feed-forward. Cấu trúc này đặc biệt hữu ích khi toàn bộ trọng số phải học từ đầu [1, 5]. So với Post-Norm, nơi LayerNorm nằm sau sublayer, Pre-Norm giữ đường residual gần đồng nhất hơn,

giúp gradient truyền qua nhiều tầng ổn định hơn và thường ít nhảy với learning rate. Đánh đổi là encoder và decoder cần thêm LayerNorm cuối để chuẩn hóa đầu ra của toàn bộ chồng lớp.

Feed-forward theo vị trí. Sau attention, mỗi vị trí được biến đổi độc lập bởi cùng một mạng hai lớp:

$$\text{FFN}(u) = W_2 \text{GELU}(W_1 u + b_1) + b_2, \quad (20)$$

trong đó W_1 mở rộng từ 384 lên 1.536 chiều và W_2 chiều ngược về 384 chiều. Attention trộn thông tin giữa các vị trí; FFN tái tổ hợp đặc trưng trong từng vị trí. GELU tạo cổng trơn dựa trên độ lớn đầu vào và thường được viết gần đúng bởi [7]

$$\xi(u) = \sqrt{\frac{2}{\pi}} (u + 0,044715u^3), \quad (21)$$

$$\text{GELU}(u) \approx \frac{u}{2} [1 + \tanh(\xi(u))]. \quad (22)$$

Khối decoder. Một lớp decoder có ba sublayer. Gọi $\mathbf{Z}^{(\ell-1)}$ là trạng thái decoder và $\mathbf{H}$ là memory cuối của encoder:

$$\mathbf{U}^{(\ell)} = \mathbf{Z}^{(\ell-1)} + \mathcal{D}[\text{MHA}(\text{LN}_1(\mathbf{Z}^{(\ell-1)}); M_{\text{tgt}})], \quad (23)$$

$$Q_c = \text{LN}_2(\mathbf{U}^{(\ell)}), \quad K_c = V_c = \mathbf{H}, \quad (24)$$

$$\mathbf{V}^{(\ell)} = \mathbf{U}^{(\ell)} + \mathcal{D}[\text{MHA}(Q_c, K_c, V_c; M_{\text{src}})], \quad (25)$$

$$\mathbf{Z}^{(\ell)} = \mathbf{V}^{(\ell)} + \mathcal{D}[\text{FFN}(\text{LN}_3(\mathbf{V}^{(\ell)}))], \quad (26)$$

trong đó $\mathcal{D}$ ký hiệu dropout và $\text{LN}_1, \text{LN}_2, \text{LN}_3$ là ba LayerNorm học được của một decoder layer. Memory $\mathbf{H}$ đã được chuẩn hóa bởi final LayerNorm của encoder, vì vậy decoder không sử dụng thêm một LayerNorm riêng trên $\mathbf{H}$ trong từng lớp cross-attention. Sublayer thứ nhất là masked self-attention và chỉ tổng hợp tiền tố đã sinh. Sublayer thứ hai là cross-attention: truy vấn đến từ decoder,

còn khóa và giá trị đến từ encoder. Vì vậy, tại mỗi bước sinh, decoder có thể chọn lại phần bài báo liên quan với tiền tố hiện tại thay vì nén toàn bộ nguồn vào một vector cố định.

Causal mask và xác suất đầu ra. Causal mask được định nghĩa

$$(M_{\text{causal}})_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (27)$$

Mask này được hợp với target padding mask. Sau lớp decoder cuối, phép chiếu ra vocab và softmax cho

$$\mathbf{o}_t = \text{LN}(\mathbf{z}_t^{(L_d)}) E^\top, \quad \mathbf{o}_t \in \mathbb{R}^{|V|}, \quad (28)$$

$$P(y_t = k \mid y_{<t}, \mathbf{x}) = \frac{\exp(o_{t,k})}{\sum_{j=1}^{|V|} \exp(o_{t,j})}. \quad (29)$$

Trong huấn luyện, toàn bộ vị trí đích được tính song song nhờ causal mask. Trong suy luận, token dự đoán ở bước t trở thành một phần đầu vào ở bước $t + 1$, nên quá trình sinh vẫn tự hồi quy.

C. Chỉ số đánh giá

ROUGE-1 và ROUGE-2 đo mức trùng unigram và bigram; ROUGE-L dựa trên dãy con chung dài nhất [2]. Compression ratio được định nghĩa

$$\text{CR} = \frac{\text{\#từ trong prediction}}{\text{\#từ trong source}} \times 100\%. \quad (30)$$

Điểm overall của LLM judge là

$$S = 0.35F + 0.25C + 0.15R + 0.15L + 0.10N, \quad (31)$$

trong đó F, C, R, L và N lần lượt biểu diễn độ trung thực, độ bao phủ, mức tập trung, độ trôi chảy và độ

sức tích. Major factual error được báo cáo riêng để tránh phạt lỗi factual hai lần.

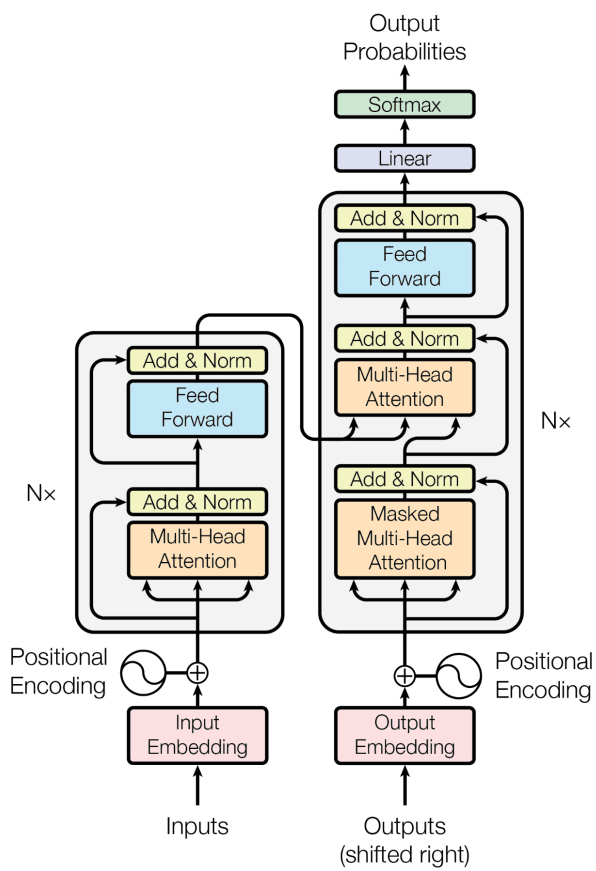


Fig. 2. Kiến trúc Transformer minh họa (dự án này dùng pre-norm).

III. DỮ LIỆU VÀ TIỀN XỬ LÝ

Sau bước làm sạch, kho dữ liệu gồm 246.975 cặp bài báo–tóm tắt. Sau khi khử trùng lặp, 197.580 mẫu, đúng 80% kho sạch, được phân vào train. Phần 20% còn lại gồm 49.395 mẫu tạo thành pool ngoài train; từ pool này, nghiên cứu chọn ngẫu nhiên 2.000 mẫu cho validation và 2.000 mẫu cho test. Như vậy, tập thực nghiệm được báo cáo gồm 201.580 mẫu, còn 45.395 mẫu trong pool không được sử dụng do nghiên cứu giới hạn quy mô validation và test ở 2.000 mẫu mỗi tập để kiểm soát chi phí suy luận và đánh giá. Trình tự là khử trùng lặp, tạo train và pool ngoài train, rồi chọn validation và test từ pool. Quy trình làm sạch gồm: chuẩn hóa Unicode NFC; loại HTML, URL, ký tự điều khiển và ký tự nhiễu; chuẩn hóa khoảng trắng; giữ bài báo dài 100–1.200 từ và tóm tắt dài 10–150 từ; dùng mô hình embedding paraphrase-multilingual-mpnet-base-v2 [?, ?] để tính cosine similarity và chỉ giữ các cặp source–reference có độ tương đồng từ 0,50 trở lên. Trùng lặp được xử lý trước khi chọn ngẫu nhiên validation và test.

TABLE I. Thống kê mô tả dữ liệu sạch.

Chỉ số	Bài báo	Tóm tắt
Trung bình (từ)	467,6	41,3
Độ lệch chuẩn	217,5	15,8
Trung vị (từ)	432,0	38,0
Min–Max (từ)	100–1.200	10–150

Compression ratio trung bình của reference là 10,62%, tương đương tóm tắt dài khoảng 1/9,4 bài gốc. Cosine similarity trung bình là 0,706, cho thấy source và reference có mức căn chỉnh ngữ nghĩa tương đối tốt sau lọc. Token fertility trung bình xấp xỉ 1,14 và trung vị xấp xỉ 1,13, phù hợp với dữ



Fig. 3. Quy trình làm sạch và chuẩn hóa dữ liệu.

liệu báo điện tử và cho thấy phần lớn từ tiếng Việt được biểu diễn gần một token. Các ngưỡng 1.200 từ cho source và 150 từ cho reference thuộc bước làm sạch ở cấp từ. Chúng khác với giới hạn ở cấp token của Scratch Transformer: source dài được cắt xuống tối đa 768 token tính cả EOS, còn mẫu có reference vượt 95 token nội dung bị loại khỏi dữ liệu dùng cho Scratch thay vì cắt cụt reference.

TABLE II. Phân chia dữ liệu thực nghiệm.

Tập	Số mẫu	Vai trò
Train	197.580	Huấn luyện
Validation	2.000	Chọn cấu hình
Test	2.000	Đánh giá cuối

Validation chỉ dùng để lựa chọn cấu hình; test chỉ được sử dụng sau khi cấu hình đã đóng băng. Mỗi mẫu có ID ổn định, prediction được ghép với source và reference theo ID thay vì thứ tự dòng.

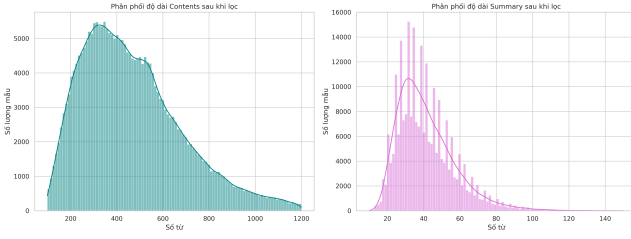


Fig. 4. Phân phối độ dài bài báo và tóm tắt.

IV. CÁC HỆ THỐNG SO SÁNH

A. Lead baselines

Lead-1 và Lead-3 dùng cùng hàm tách câu có bảo vệ tên miền, ngày tháng, số thập phân, chức danh và các trường hợp như TP.HCM. Vì tiền xử lý giống nhau, khác biệt giữa hai baseline chủ yếu phản ánh số câu được lấy.

B. Scratch Transformer

Mục tiêu thiết kế. Scratch Transformer được xây dựng như một mô hình encoder-decoder thuần túy trong PyTorch, không nạp trọng số tiền huấn luyện và không gọi trực tiếp lớp nn.Transformer. Các thành phần Token Embedding, Sinusoidal Positional Encoding, Multi-Head Attention, Position-wise Feed-Forward, Encoder Layer, Decoder Layer và mô hình Seq2Seq được cài đặt thành các module riêng. Cách tổ chức này làm rõ luồng tensor và mask, cho phép kiểm soát chi tiết quá trình huấn luyện, đồng thời tạo một đối chứng về năng lực có thể học được chỉ từ 197.580 cặp train.

Bước 1: huấn luyện tokenizer và quy ước token.

Tokenizer SentencePiece Unigram học trực tiếp từ chuỗi Unicode đã chuẩn hóa và tạo vocabulary 16.000 subword [4]. Bốn token đặc biệt được cố định là PAD=0, UNK=1, BOS=2 và EOS=3. Sentence-

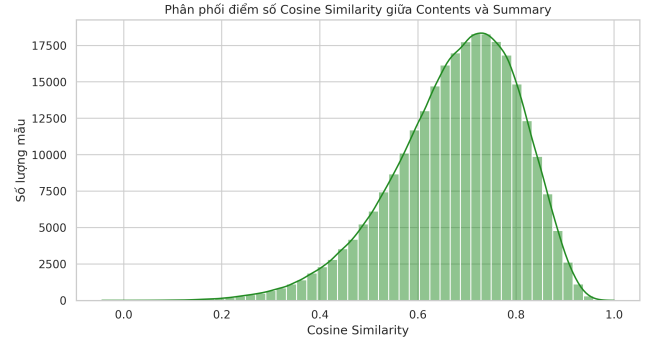


Fig. 5. Phân phối cosine similarity giữa source và reference. (trước khi loại bỏ các mẫu có cosine similarity < 0.5)

TABLE III. Cấu hình Scratch Transformer.

Thành phần	Giá trị
Kiến trúc	Encoder-Decoder, Pre-Norm
Số lớp	6 encoder + 6 decoder
$d_{\text{model}} / d_{\text{ff}}$	384 / 1.536
Attention heads	6 ($d_k = 64$)
Tokenizer	SPM Unigram, vocab 16.000
Độ dài source/target	768 / 96 token
Dropout / label smoothing	0,1 / 0,05
Activation	GELU
Embedding	Chia sẻ và tied output

Piece chọn phân đoạn có xác suất cao dưới mô hình unigram, nhờ đó có thể biểu diễn cả từ hiếm bằng các mảnh đã biết thay vì mở rộng vocabulary theo từng từ. Số token trung bình là 533,3 cho source và 46,6 cho target; tên riêng, từ ngoại lai, chuỗi số và ký hiệu thường bị tách thành nhiều subword.

Gọi $(s_1, \dots, s_n)$ là token nguồn và $(r_1, \dots, r_m)$ là token reference sau SentencePiece. Phần lớn mẫu nằm trong vùng độ dài mà mô hình có thể xử lý. Source vượt giới hạn được cắt bằng cách giữ tối đa 767 token nội dung rồi thêm EOS:

$$\mathbf{x} = [s_1, \dots, s_{\min(n, 767)}, \text{EOS}]. \quad (32)$$

TABLE IV. Lý do lựa chọn các thành phần thiết kế của Scratch Transformer.

Lựa chọn	Mục đích	Đánh đổi hoặc giới hạn
Encoder–Decoder	Mã hóa toàn bộ bài báo rồi sinh một chuỗi có độ dài khác nguồn	Cross-attention làm tăng chi phí nhưng phù hợp tự nhiên với bài toán seq2seq
Pre-Norm	Giữ residual path ổn định và hỗ trợ truyền gradient qua 12 lớp	Phải có LayerNorm cuối encoder và decoder
$d = 384$, 6 head	Mỗi head có 64 chiều; cân bằng năng lực và giới hạn bộ nhớ GPU	Nhỏ hơn các mô hình pretrained base, nên năng lực ngôn ngữ nền hạn chế hơn
$d_{ff} = 4d$	Tạo không gian phi tuyến rộng sau bước trộn ngữ cảnh	FFN chiếm phần lớn tham số của mỗi layer
Embedding chia sẻ và tied output	Đồng nhất không gian token nguồn–đích, giảm mạnh số tham số	Chỉ phù hợp khi encoder và decoder dùng chung vocabulary
Sinusoidal position	Không thêm tham số học và hỗ trợ vị trí tới giới hạn cấu hình	Có thể kém linh hoạt hơn positional representation hiện đại
Source 768, target 96	Bao phủ phần lớn phân phối token với chi phí khả thi	Source vượt giới hạn bị cắt; mẫu có reference vượt 95 token nội dung bị loại khỏi dữ liệu Scratch

Đối với target, nghiên cứu không cắt reference dài xuống 95 token vì thao tác đó có thể tạo nhãn bị cắt. Thay vào đó, mẫu có $m > 95$ token nội dung reference bị loại trước khi tạo các mini-batch huấn luyện Scratch Transformer. Số mẫu bị loại bởi điều kiện token này là 3739 mẫu train bị loại, 33102 mẫu train bị cắt source. Với các mẫu còn lại, $m \leq 95$ và

$$\mathbf{y}^{\text{in}} = [\text{BOS}, r_1, \dots, r_m], \quad (33)$$

$$\mathbf{y}^{\text{label}} = [r_1, \dots, r_m, \text{EOS}]. \quad (34)$$

Do đó, source có tối đa 768 token tính cả EOS; target có tối đa 95 token nội dung và 96 vị trí sau khi thêm BOS hoặc EOS. Cặp $\mathbf{y}^{\text{in}} - \mathbf{y}^{\text{label}}$ dịch nhau một vị trí, đúng với nhiệm vụ dự đoán token kế tiếp.

Padding được thực hiện động theo chuỗi dài nhất trong từng mini-batch thay vì luôn đệm tới 768/96. Với batch B , source được đệm tới S_B và target tới T_B ; cách này giảm số phép tính trên PAD khi batch chứa nhiều chuỗi ngắn. ID gốc của mẫu được giữ

song song với tensor để prediction sau cùng có thể ghép lại theo ID.

Bước 2: embedding, vị trí và chia sẻ trọng số.

Mô hình tạo một ma trận embedding duy nhất

$$E \in \mathbb{R}^{16000 \times 384}. \quad (35)$$

Encoder và decoder cùng tra cứu từ E . Trước khi đi qua layer đầu, vector embedding được nhân $\sqrt{384}$, cộng positional encoding theo (3) rồi áp dụng dropout 0,1. Ma trận E đồng thời được dùng làm trọng số chiếu đầu ra:

$$\mathbf{o}_t = \text{LN}(\mathbf{z}_t^{(L_d)})E^\top, \quad \mathbf{o}_t \in \mathbb{R}^{16000}. \quad (36)$$

Đây là *weight tying* [?]: thay vì học thêm ma trận 384×16000 , mô hình đo độ phù hợp giữa trạng thái decoder và chính các vector token đã dùng ở đầu vào. Tying vừa giảm tham số vừa buộc không gian biểu diễn đầu vào và đầu ra có cùng hình học. Trong cài đặt này không có ma trận output độc

lập; positional encoding được đăng ký như buffer, không nhận gradient.

Bước 3: cài đặt Multi-Head Attention. Mỗi attention module chứa bốn phép chiếu tuyến tính $W^Q, W^K, W^V, W^O \in \mathbb{R}^{384 \times 384}$, có bias. Sau phép chiếu, tensor được đổi hình dạng và chuyển trục:

$$[B, L, 384] \rightarrow [B, L, 6, 64] \rightarrow [B, 6, L, 64]. \quad (37)$$

Tích $QK^\top$ vì thế tạo score có dạng $[B, 6, L_q, L_k]$. Mask được broadcast tới đúng dạng score, các phần tử bị cấm được gán $-\infty$ trước softmax. Sau khi nhân với V , sáu head được chuyển về $[B, L, 6, 64]$, nối thành $[B, L, 384]$ và đi qua W^O . Việc kiểm tra thứ tự các chiều ở đây quan trọng: đảo nhầm trục head và sequence vẫn có thể chạy nhưng làm attention học sai cấu trúc.

Bước 4: tạo và kết hợp mask. Mô hình sử dụng ba nguồn mask khác nhau. Source padding mask chặn các cột khóa tương ứng PAD trong cả encoder self-attention và cross-attention. Target padding mask chặn token đệm ở decoder. Causal mask chặn phần tam giác phía trên để vị trí i không nhìn thấy $j > i$. Hai mask target được hợp bằng phép OR logic rồi broadcast trên batch và head.

Không cần tạo một bản mask riêng cho từng head: broadcasting giúp sáu head dùng chung quy tắc hợp lệ. Source mask chỉ chặn phía key; biểu diễn tại vị trí query bị đệm không ảnh hưởng loss vì label PAD bị bỏ qua. Khi cross-attention, $L_q = T$ và $L_k = S$, do đó causal mask không được dùng; decoder được phép đọc mọi token nguồn không phải padding.

Bước 5: ghép encoder. Sáu encoder layer có cùng cấu trúc nhưng không chia sẻ trọng số. Trong mỗi layer, đầu vào lần lượt đi qua Pre-LN

self-attention, residual-dropout, Pre-LN FFN và residual-dropout như (16)–(18). Self-attention là hai chiều, vì thế biểu diễn của một token có thể dùng ngữ cảnh trước và sau nó. Sau layer thứ sáu, final LayerNorm tạo memory ổn định cho decoder. Encoder chỉ được tính một lần cho mỗi source trong suy luận.

Bước 6: ghép decoder. Sáu decoder layer cũng có trọng số riêng. Masked self-attention mô hình hóa quan hệ nội bộ của tiền tố tóm tắt; cross-attention nối tiền tố đó với bài báo; FFN biến đổi phi tuyến từng vị trí. Ở cross-attention, Q đến từ trạng thái decoder, trong khi K, V đến từ memory encoder. Sự phân vai này có diễn giải trực tiếp: câu hỏi “cần lấy thông tin gì tiếp theo” được biểu diễn bởi Q , còn “nguồn đang cung cấp thông tin nào” được biểu diễn bởi K, V . Final LayerNorm và phép chiếu tied trong (36) tạo logits.

Luồng forward khi huấn luyện có thể tóm tắt thành sáu bước:

1. tạo source IDs, shifted target IDs và ba loại mask;
2. mã hóa source thành memory $\mathbf{H}$;
3. mã hóa toàn bộ $\mathbf{y}^{\text{in}}$ song song nhờ causal mask;
4. tại mỗi decoder layer, dùng cross-attention để truy vấn $\mathbf{H}$;
5. chiếu trạng thái decoder sang 16.000 logits bằng embedding đã tied;
6. so logits với $\mathbf{y}^{\text{label}}$ tại các vị trí không phải PAD.

Phân rã số tham số. Với $d = 384$, $d_{\text{ff}} = 1536$ và $V = 16000$, shared embedding có

$$N_{\text{emb}} = Vd = 16000 \times 384 = 6.144.000. \quad (38)$$

TABLE V. Kích thước tensor chính trong một forward pass.

Tensor	Kích thước	Ý nghĩa
Source/target IDs	$[B, S], [B, T]$	ID SentencePiece sau dynamic padding
Source/target embedding	$[B, S, 384], [B, T, 384]$	Token embedding cộng positional encoding
Q, K, V trước khi tách head	$[B, L, 384]$	Kết quả ba phép chiếu tuyến tính
Q, K, V sau khi tách head	$[B, 6, L, 64]$	Sáu không gian attention song song
Encoder self-attention score	$[B, 6, S, S]$	Mỗi vị trí nguồn truy vấn toàn bộ nguồn hợp lệ
Decoder self-attention score	$[B, 6, T, T]$	Bị chặn bởi causal và target padding mask
Cross-attention score	$[B, 6, T, S]$	Mỗi vị trí đích truy vấn memory nguồn
Encoder memory	$[B, S, 384]$	Đầu ra sau 6 encoder layer và final Layer-Norm
Decoder logits	$[B, T, 16000]$	Logit cho toàn bộ vocabulary tại mỗi vị trí

TABLE VI. Mask được dùng trong Scratch Transformer.

Mask	Dạng broadcast	Vị trí sử dụng
Source padding	$[B, 1, 1, S]$	Encoder self-attention; decoder cross-attention
Causal	$[1, 1, T, T]$	Decoder self-attention
Target padding	$[B, 1, 1, T]$	Decoder self-attention
Target hợp	$[B, 1, T, T]$	Causal OR target padding

Một MHA gồm bốn phép chiếu có bias:

$$N_{\text{MHA}} = 4(d^2 + d) = 591.360. \quad (39)$$

Một FFN có

$$N_{\text{FFN}} = (d d_{\text{ff}} + d_{\text{ff}}) + (d_{\text{ff}} d + d) \quad (40)$$

$$= 1.181.568. \quad (41)$$

Mỗi LayerNorm có $2d = 768$ tham số scale và bias.

Vì thế, một encoder layer có

$$591.360 + 1.181.568 + 2(768) = 1.774.464 \quad (42)$$

tham số; một decoder layer có hai MHA nên có

$$2(591.360) + 1.181.568 + 3(768) = 2.366.592. \quad (43)$$

TABLE VII. Số tham số học duy nhất của Scratch Transformer.

Khối	Số tham số
Shared/tied embedding	6.144.000
6 encoder layer + final LN	10.647.552
6 decoder layer + final LN	14.200.320
Tổng duy nhất	30.991.872

Con số khoảng 31 triệu là số bậc tự do được tối ưu và cũng là quy mô đúng để so sánh năng lực mô hình. Nếu cộng kích thước tensor theo mọi khóa trong state_dict, có thể nhận 49.755.648 phần tử vì cùng ma trận embedding/tied output xuất hiện dưới nhiều tên tham chiếu. Cụ thể, ba alias bổ sung đóng góp $3 \times 6.144.000$ phần tử và hai positional

buffer đóng góp $768 \times 384 + 96 \times 384 = 331.776$ phần tử. Các tensor lặp và buffer không tạo thêm tham số học; do đó không nên nói mô hình có gần 50 triệu tham số độc lập. Cụ thể, con số 49,76 triệu trên slide là tổng số phần tử khi duyệt mọi khóa của `state_dict`; con số 30.991.872 mới là số tham số học duy nhất dùng để mô tả quy mô tối ưu hóa của mô hình.

Chi phí tính toán. Encoder self-attention có độ phức tạp xấp xỉ $O(S^2d)$ và phải lưu score $[B, h, S, S]$; decoder trong huấn luyện cần $O(T^2d)$ cho self-attention và $O(TSd)$ cho cross-attention. FFN có chi phí $O((S + T)d d_{\text{ff}})$. Vì S có thể đạt 768 còn T chỉ 96, encoder self-attention và encoder FFN là các phần tiêu tốn bộ nhớ chính. Đây là lý do dynamic padding, FP16 và giới hạn độ dài nguồn có ảnh hưởng lớn tới tốc độ huấn luyện.

Suy luận tự hồi quy và beam search. Khi sinh tóm tắt, mô hình chạy encoder một lần, khởi tạo decoder bằng BOS, rồi lặp lại các bước sau: tính log-probability ở vị trí cuối; mở rộng mỗi giả thuyết bằng các token có điểm cao; giữ bốn giả thuyết tốt nhất; dừng giả thuyết gặp EOS; và kết thúc khi mọi beam hoàn tất hoặc đạt 96 token. PAD và BOS bị cấm ở đầu ra; no-repeat trigram loại các mở rộng làm lặp lại một trigram đã xuất hiện.

Điểm tích lũy của chuỗi y là

$$s(\mathbf{y}) = \sum_{t=1}^{|\mathbf{y}|} \log P(y_t | y_{<t}, \mathbf{x}). \quad (44)$$

Do tổng log-probability tự nhiên ưu tiên chuỗi ngắn, cấu hình cuối chuẩn hóa bằng length penalty theo họ công thức thường dùng trong giải mã neural sequence [?]

$$\text{score}(\mathbf{y}) = \frac{s(\mathbf{y})}{((5 + |\mathbf{y}|)/6)^{1,1}}. \quad (45)$$

Beam-4 tăng khả năng tìm chuỗi tốt hơn greedy nhưng cần lưu và tính bốn tiền tố. Cài đặt hiện tại tái tính decoder trên toàn bộ tiền tố ở mỗi bước, chưa có key-value cache. Cách này đơn giản và ít rủi ro sai cache, nhưng làm suy luận chậm hơn một cài đặt có cache.

Pipeline prediction chia dữ liệu theo tiến trình GPU, ghi shard JSONL có ID và trạng thái, rồi ghép lại theo ID. Ghi tệp nguyên tử và cơ chế tiếp tục từ shard đã hoàn thành giúp hạn chế mất kết quả khi phiên Kaggle bị ngắt. Nếu một batch gặp lỗi, pipeline có thể hạ xuống xử lý từng mẫu để lỗi cục bộ không làm hỏng toàn bộ tập test.

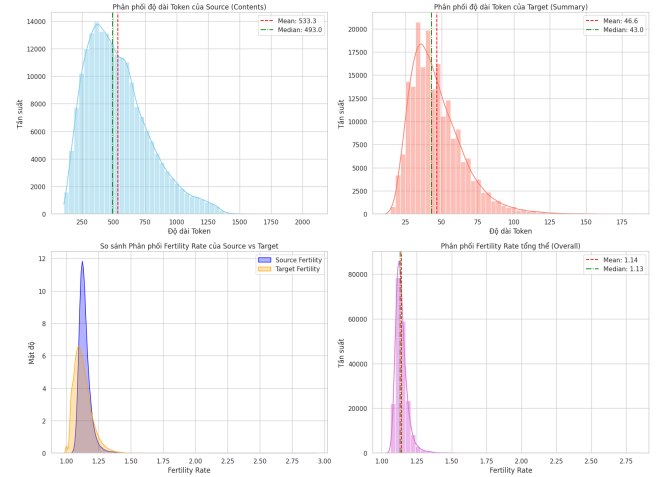


Fig. 6. Phân phối độ dài token và fertility của Sentence-Piece Unigram.

C. Huấn luyện Scratch Transformer

Teacher forcing và tính song song. Trong huấn luyện, decoder nhận toàn bộ chuỗi $\mathbf{y}^{\text{in}}$ được tạo từ reference, thay vì nhận token do chính mô hình dự đoán ở bước trước. Cơ chế này gọi là teacher forcing. Causal mask vẫn bắt buộc vị trí t chỉ dùng $y_{<t}$, nhưng tất cả vị trí được tính trong một forward pass song song. Nếu bỏ causal mask, mô hình có thể đọc chính nhãn tương lai và thu được loss thấp giả tạo mà không học được khả năng sinh tự hồi quy.

Hàm mất mát. Gọi $\mathcal{T} = \{(b, t) : y_{b,t} \neq \text{PAD}\}$ là tập vị trí target hợp lệ và $p_{b,t,k}$ là xác suất token k . Với label smoothing $\varepsilon = 0,05$, phân phối nhãn khái niệm được viết

$$q_{b,t,k} = (1 - \varepsilon)\mathbb{1}[k = y_{b,t}] + \frac{\varepsilon}{|V|}. \quad (46)$$

Cross-entropy trung bình là

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{T}|} \sum_{(b,t) \in \mathcal{T}} \sum_{k=1}^{|V|} q_{b,t,k} \log p_{b,t,k}. \quad (47)$$

PAD được đặt làm `ignore_index`, nên các token đệm không tác động đến gradient. Label smoothing ngăn mô hình đẩy xác suất của một token về gần 1 tuyệt đối, cải thiện hiệu chuẩn và giảm overconfidence [?]; đổi lại, loss không thể tiến về 0 ngay cả khi dự đoán rất chắc chắn.

Để mô tả thang của validation loss, nghiên cứu báo cáo exponentiated smoothed cross-entropy tại các token hợp lệ:

$$\text{ESCE} = \exp(\mathcal{L}_{\text{val}}^{\text{smooth}}). \quad (48)$$

Do $\mathcal{L}_{\text{val}}^{\text{smooth}}$ sử dụng label smoothing $\varepsilon = 0,05$, ESCE không hoàn toàn tương đương perplexity chuẩn được tính từ negative log-likelihood với nhãn one-hot. Muốn báo cáo perplexity chuẩn, cần chạy thêm validation với label smoothing bằng 0 chỉ cho mục đích đánh giá. Chỉ số loss này đo mức bất định token-level, không trực tiếp đo chất lượng tóm tắt ở cấp chuỗi; vì vậy checkpoint được theo dõi bằng loss nhưng cấu hình decoding cuối vẫn được lựa chọn bằng ROUGE và CR trên validation.

Tối ưu AdamW. Mô hình được tối ưu bằng AdamW với $(\beta_1, \beta_2) = (0,9, 0,98)$, $\epsilon = 10^{-9}$ và

weight decay $\lambda = 0,01$ [6]. Với gradient g_t , các moment được cập nhật

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (49)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (50)$$

sau đó hiệu chỉnh bias thành $\hat{m}_t, \hat{v}_t$. Dạng cập nhật rút gọn của AdamW là

$$\theta_{t+1} = (1 - \eta_t \lambda) \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}. \quad (51)$$

Weight decay được tách khỏi gradient của loss, khác với việc cộng regularization L_2 trực tiếp vào gradient Adam.

Warmup và cosine decay. Learning rate cực đại là $\eta_{\text{max}} = 3 \times 10^{-4}$, learning rate cuối là $\eta_{\text{min}} = 10^{-6}$ và warmup kéo dài $T_w = 3.030$ bước. Đặt

$$r_t = \frac{t - T_w}{T_{\text{max}} - T_w}, \quad c_t = \frac{1 + \cos(\pi r_t)}{2}. \quad (52)$$

Trong warmup, $\eta_t = \eta_{\text{max}} t / T_w$ với $0 \leq t \leq T_w$. Sau warmup, lịch cosine là

$$\eta_t = \eta_{\text{min}} + (\eta_{\text{max}} - \eta_{\text{min}}) c_t, \quad T_w < t \leq T_{\text{max}}. \quad (53)$$

Warmup tránh cập nhật quá lớn khi các moment của AdamW và activation của mô hình còn chưa ổn định. Sau warmup, cosine decay chuyển từ học nhanh sang tinh chỉnh nhỏ dần quanh nghiệm tốt.

Gradient clipping. Sau khi backpropagation, toàn bộ gradient được chuẩn hóa nếu norm vượt ngưỡng $c = 1,0$:

$$g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|_2}\right). \quad (54)$$

Clipping không thay đổi hướng gradient, chỉ giảm độ lớn của những bước bất thường. Đây là lớp bảo

vệ quan trọng với chuỗi dài và attention score có thể dao động mạnh ở giai đoạn đầu.

Mixed precision và huấn luyện phân tán. Hai GPU T4 chạy Distributed Data Parallel; mỗi GPU nhận batch cục bộ 32 nên batch toàn cục là $2 \times 32 = 64$. Mỗi tiến trình tính forward và backward trên shard riêng, sau đó DDP all-reduce gradient để mọi bản sao mô hình áp dụng cùng một optimizer step. Sampler phân tán được đổi seed theo epoch để các shard được xáo trộn nhất quán mà không lặp mẫu giữa GPU.

FP16 autocast dùng số thực 16 bit cho các phép nhân ma trận phù hợp, trong khi các phép nhảy cảm và master weights vẫn được quản lý ở độ chính xác an toàn. Dynamic loss scaling nhân loss trước backward để tránh gradient nhỏ bị underflow; trước clipping, gradient được unscale. Thứ tự một training step là:

1. xóa gradient và chuyển batch tới GPU;
2. chạy forward dưới autocast và tính loss trong (47);
3. scale loss, backward và all-reduce gradient qua DDP;
4. unscale gradient rồi clip global norm tại 1,0;
5. gọi AdamW step, cập nhật loss scaler và learning-rate scheduler.

Thứ tự unscale trước clipping là bắt buộc; nếu clip gradient còn đang được scale, ngưỡng 1,0 không còn ý nghĩa.

Validation và checkpoint. Sau mỗi epoch, mô hình chuyển sang eval để tắt dropout và tính loss trên toàn bộ validation mà không cập nhật gradient. Checkpoint chỉ thay thế khi validation loss giảm;

TABLE VIII. Cấu hình tối ưu Scratch Transformer.

Siêu tham số	Giá trị
Epoch tối đa / patience	20 / 4
GPU / batch mỗi GPU	2 / 32
Batch toàn cục	64
Optimizer	AdamW
$\beta_1, \beta_2, \epsilon$	0,9; 0,98; 10^{-9}
LR cực đại / cực tiểu	$3 \times 10^{-4} / 10^{-6}$
Warmup	3.030 bước
Weight decay	0,01
Gradient clip	1,0
Precision	AMP FP16

trạng thái cần lưu gồm trọng số mô hình, optimizer, scheduler, AMP scaler, epoch và giá trị tốt nhất để có thể tiếp tục đúng lịch học. Mô hình được huấn luyện tối đa 20 epoch với early-stopping patience 4. Checkpoint tốt nhất xuất hiện ở epoch 17 với validation loss có label smoothing là 2,5539. Giá trị exponentiated smoothed cross-entropy tương ứng là

$$\text{ESCE} = \exp(2,5539) \approx 12,86. \quad (55)$$

Giá trị 12,86 không được diễn giải là perplexity one-hot chuẩn. Các epoch sau không tạo checkpoint tốt hơn; vì vậy checkpoint epoch 17, không phải trạng thái epoch cuối, được dùng cho bước lựa chọn decoding. Cơ chế patience bảo vệ khỏi việc tiếp tục huấn luyện lâu sau khi validation ngừng cải thiện, còn giới hạn 20 epoch đặt trần chi phí thực nghiệm.

Khác biệt giữa train và inference. Trong train, mô hình luôn nhận tiền tố đúng từ reference; trong inference, nó nhận lại token do chính nó sinh. Một lỗi sớm có thể làm tiền tố lệch khỏi phân phối train và kéo theo lỗi ở các bước sau, thường gọi là exposure bias. Beam search và no-repeat trigram cải thiện tìm kiếm nhưng không sửa được sai lệch phân

phối này. Đây là một nguyên nhân khiến token-level validation loss tốt chưa bảo đảm factuality tốt, đặc biệt với tên riêng, số liệu và quan hệ sự kiện.

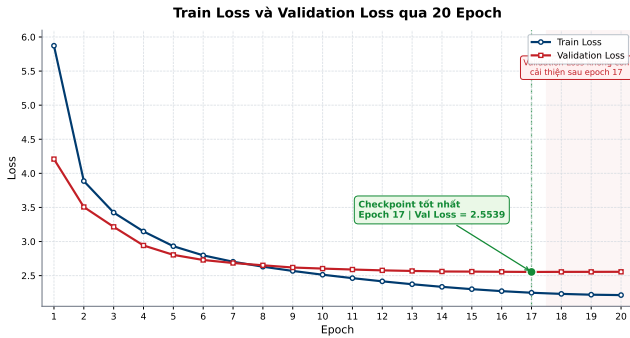


Fig. 7. Train loss và validation loss trong quá trình huấn luyện.

D. ViT5

ViT5 sử dụng checkpoint VietAI/vit5-base-vietnews-summarization, một checkpoint ViT5-Base đã được fine-tune cho bài toán tóm tắt tin tức tiếng Việt, chứ không chỉ là một mô hình nền được tiền huấn luyện [?, 8]. Kiến trúc ViT5 kế thừa khuôn khổ text-to-text của T5 [3].

Source được đưa trực tiếp vào tokenizer mà không bổ sung task prefix. Tất cả cấu hình khảo sát đều sử dụng seed 42, beam search với bốn beam, max_new_tokens=128, no_repeat_ngram_size=3 và do_sample=False. Biến được khảo sát trên tập validation là length penalty trong tập {0,8; 1,0; 1,1}. Dựa trên kết quả ROUGE và compression ratio, nhóm lựa chọn cấu hình Beam-4, LP=1,1, max128, nr=3 và cố định cấu hình giải mã này trước khi thực hiện đánh giá cuối trên tập test.

ViT5 và Scratch Transformer được huấn luyện trên hai tập dữ liệu khác nhau nhưng cùng thuộc miền tin tức tiếng Việt. Hai hệ thống được đánh giá trên cùng tập validation và test, sử dụng cùng source, reference, ID, quy trình ghép prediction và

bộ chỉ số đánh giá. Do đó, kết quả cho phép so sánh chất lượng của hai hệ thống tóm tắt trong cùng miền ứng dụng.

V. THIẾT KẾ THỰC NGHIỆM

Thiết kế thực nghiệm được tổ chức quanh bốn câu hỏi bổ sung: (i) Scratch Transformer vượt các base-line trích xuất đến đâu; (ii) mô hình huấn luyện từ đầu khác ViT5 tiền huấn luyện như thế nào; (iii) lựa chọn giải mã ảnh hưởng ra sao đến ROUGE và độ dài tóm tắt; và (iv) kết luận trên validation có giữ được trên test độc lập hay không.

Mỗi hệ thống sinh được thử trên validation trước khi chạy test. Các biến được khảo sát gồm greedy hoặc beam search, beam size 4, length penalty trong {0,8; 1,0; 1,1}, chặn lặp trigram và giới hạn sinh 128 token cho ViT5. Quyết định ưu tiên ROUGE-1/2/L F1, đồng thời kiểm tra compression ratio để tránh chọn một cấu hình chỉ vì sinh dài hơn.

Scratch Transformer chọn Beam-4, LP=1,1, nr=3 vì dẫn đầu R-1 và R-2, dù LP=0,8 nhỉnh hơn 0,0346 điểm R-L. So với greedy, cấu hình được chọn tăng 1,0979 điểm R-1 và 1,2979 điểm R-2. Việc CR tăng từ 8,7365% lên 9,7061% cho thấy một phần lợi thế ROUGE đi kèm đầu ra dài hơn, nên quyết định không dựa trên ROUGE đơn lẻ. ViT5 chọn beam4, LP=1,1, max128, nr=3 vì dẫn đầu R-1 và R-L; R-2 chỉ thấp hơn LP=1,0 đúng 0,0057 điểm. Từ LP=0,8 đến 1,1, R-1 chỉ tăng 0,1406 điểm và CR tăng nhẹ từ 8,7852% lên 8,9242%, cho thấy ViT5 tương đối ít nhạy với length penalty trong khoảng khảo sát. Sau lựa chọn, hai cấu hình được đóng băng trước khi chạy test. Quy tắc này trực tiếp kiểm tra liệu lợi thế của Scratch trên validation có bền vững trên một tập test độc lập hay không và ngăn việc chọn cấu hình

TABLE IX. Toàn bộ cấu hình được đánh giá trên validation.

Hệ thống	Cấu hình	R-1	R-2	R-L	CR (%)
Scratch	Greedy	33,0618	16,9446	27,5080	8,7365
Scratch	Beam-4, LP=0,8, nr=3	33,7884	18,2143	28,4245	8,4232
Scratch	Beam-4, LP=1,0, nr=3	33,9924	18,1839	28,3486	9,2057
Scratch	Beam-4, LP=1,1, nr=3	34,1597	18,2425	28,3899	9,7061
ViT5	beam4_lp0.8_max128_nr3	31,1089	14,1935	24,6530	8,7852
ViT5	beam4_lp1.0_max128_nr3	31,2016	14,2340	24,6987	8,8863
ViT5	beam4_lp1.1_max128_nr3	31,2495	14,2283	24,7108	8,9242
Baseline	Lead-1	25,6098	11,0876	19,6637	11,0935
Baseline	Lead-3	24,9105	10,6417	17,5409	29,7882

theo điểm test. Smoke test 10 mẫu chỉ kiểm tra pipeline, schema và lỗi kỹ thuật, không được dùng để chọn mô hình.

Evaluator chuẩn hóa Unicode NFC, chuyển chữ thường và tách từ bằng underthesea phiên bản 9.5.0. Cả bốn hệ thống đều có đủ 2.000 prediction hợp lệ. ROUGE đo mức độ trùng khớp với reference, còn LLM judge đánh giá chất lượng trực tiếp dựa trên source. Hai phương pháp được sử dụng bổ sung cho nhau.

Reference audit nhận source và reference để kiểm tra mức độ reference được source hỗ trợ. Candidate evaluation nhận source và bốn summary nhưng không nhận reference. Tên hệ thống được ẩn; vị trí A/B/C/D được cân bằng để mỗi hệ thống xuất hiện đúng 50 lần tại mỗi vị trí trên 200 mẫu. Bốn hệ thống được chấm paired trên cùng ID; bootstrap được thực hiện theo sample ID.

VI. KẾT QUẢ

A. Đánh giá tự động

Scratch Transformer dẫn đầu cả ba chỉ số ROUGE. ViT5 đứng thứ hai và có CR thấp nhất. Lead-3 dài nhất nhưng không cải thiện ROUGE so với Lead-1, cho thấy việc thêm hai câu làm tăng lượng nội

TABLE X. Kết quả trên 2.000 mẫu test.

Hệ thống	R-1	R-2	R-L	CR (%)
Lead-1	26,78	12,17	20,80	11,51
Lead-3	25,12	11,03	17,92	30,14
Scratch Transformer	33,84	17,84	28,30	9,42
ViT5	31,22	14,51	25,00	8,63

dung nhưng đồng thời đưa vào nhiều phần không trùng reference, từ đó giảm F1.

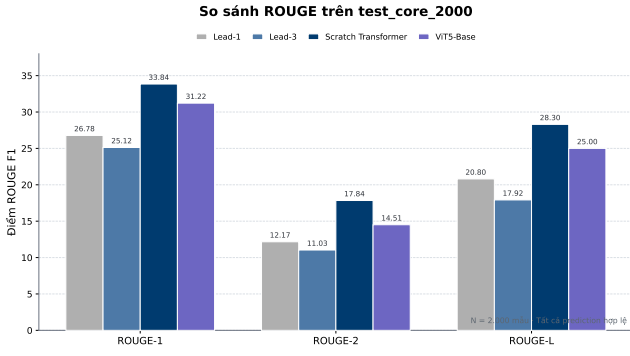


Fig. 8. ROUGE F1 của bốn hệ thống trên test core 2.000 mẫu.

B. Chất lượng reference

Trong 200 mẫu phân tầng theo độ dài source, 108 reference được hỗ trợ đầy đủ, 75 được hỗ trợ một phần, 17 phần lớn không có căn cứ và không có mẫu nào ghép sai hoàn toàn. Như vậy, 46,0% reference có ít nhất một ý cốt lõi không được source hỗ trợ. Các lỗi thường gặp là thêm ngày tháng, địa điểm, số liệu, chức danh hoặc nguồn phát biểu không có trong source. ROUGE vẫn hữu ích cho so sánh trên cùng tập, nhưng không đủ để khẳng định factuality tuyệt đối.

TABLE XI. Kết quả reference audit trên 200 mẫu.

Nhân	Số mẫu	Tỷ lệ
Fully supported	108	54,0%
Partially supported	75	37,5%
Unsupported	17	8,5%
Mismatched	0	0,0%

C. Giao thức LLM-as-a-Judge

Đánh giá LLM-as-a-Judge sử dụng rubric có cấu trúc dựa trên các nghiên cứu trước [?, ?]. Thí nghiệm được thực hiện ngày 22/08/2026 bằng **GPT-5.6 Solar Max** trên 200 source, tương ứng 800 candidate summary. Dữ liệu được chia thành 20 phiên hội thoại tách biệt, mỗi phiên gồm 10 source và 40 candidate.

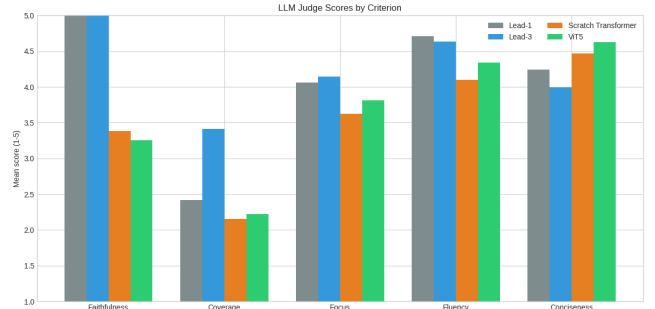


Fig. 9. Điểm LLM judge theo từng tiêu chí trên 200 source.

Với mỗi source, bốn candidate được ký hiệu A/B/C/D; tên hệ thống và reference đều được ẩn. Vị trí được cân bằng để mỗi hệ thống xuất hiện đúng 50 lần tại từng vị trí. Judge sử dụng cùng prompt, rubric, thang điểm và trọng số cho toàn bộ thí nghiệm, đồng thời chấm năm tiêu chí theo thang 1–5, điểm Overall, major factual error và nhóm lỗi factual. Bốn hệ thống được đánh giá paired trên cùng sample ID; bootstrap và so sánh từng cặp cũng được thực hiện theo sample ID.

Nhận xét. Bảng 12 cho thấy thứ hạng theo đánh giá source-grounded khác đáng kể so với thứ hạng ROUGE. Lead-3 đạt Overall cao nhất (4,32) nhờ Faithfulness tuyệt đối (5,00) và Coverage cao nhất (3,42). Tuy nhiên, lợi thế này đi kèm Conciseness thấp nhất (4,00), cho thấy kết quả của Lead-3 được hỗ trợ một phần bởi ngân sách độ dài lớn hơn. Lead-1 có Coverage thấp hơn Lead-3 nhưng vẫn đạt Overall 4,09 nhờ giữ nguyên văn bản nguồn và duy trì Fluency cao. Trong hai mô hình sinh, ViT5 nhỉnh hơn Scratch Transformer về Coverage, Focus, Fluency, Conciseness và Overall, trong khi Scratch Transformer nhỉnh hơn về Faithfulness. Chênh lệch Overall giữa hai mô hình chỉ là 0,05 điểm, vì vậy bảng này không cung cấp bằng chứng đủ mạnh để khẳng định một mô hình sinh vượt trội toàn diện.

TABLE XII. Điểm trung bình theo năm tiêu chí trên cùng 200 source.

Hệ thống	Faith.	Cover.	Focus	Fluency	Concise.	Overall
Lead-1	5,00	2,42	4,06	4,71	4,25	4,09
Lead-3	5,00	3,42	4,15	4,63	4,00	4,32
Scratch Transformer	3,39	2,16	3,62	4,10	4,47	3,33
ViT5	3,26	2,22	3,81	4,34	4,63	3,38

Tỷ lệ major factual error của Scratch Transformer là 45,5%, còn ViT5 là 52,0%. Hai mô hình sinh có Overall lần lượt là 3,328 và 3,379. Chênh lệch Scratch-ViT5 bằng $-0,052$ với bootstrap 95% CI $[-0,229; 0,125]$, do đó chưa đủ bằng chứng để kết luận hệ thống nào tốt hơn về Overall.

TABLE XIII. Compression ratio và Conciseness trên cùng 200 mẫu LLM judge.

Hệ thống	CR (%)	Conciseness
Lead-1	11,32	4,245
Lead-3	29,00	3,995
Scratch Transformer	9,42	4,470
ViT5	8,59	4,625

Nhận xét. Bảng 13 làm rõ trade-off giữa độ bao phủ và độ súc tích. Lead-3 có CR 29,00%, cao hơn đáng kể ba hệ thống còn lại, nhưng Conciseness chỉ đạt 3,995, thấp nhất trong bảng. Ngược lại, ViT5 tạo đầu ra ngắn nhất với CR 8,59% và đạt Conciseness cao nhất, 4,625. Scratch Transformer nằm giữa hai xu hướng này với CR 9,42% và Conciseness 4,470. Lead-1 có CR 11,32% nhưng vẫn giữ Conciseness 4,245 do chỉ trích xuất một câu hoàn chỉnh. CR trên 200 mẫu gần với kết quả trên 2.000 mẫu, với chênh lệch tối đa khoảng 1,15 điểm phần trăm, nên tập LLM judge có tính đại diện tương đối tốt về độ dài đầu ra. Mối liên hệ giữa CR và Conciseness trong bảng là quan sát mô tả, không được diễn giải như quan hệ nhân quả.

TABLE XIV. Các nhóm lỗi factual của mô hình sinh.

Loại lỗi	Scratch	ViT5
RELATION	30,5%	18,5%
PREDICATE	24,5%	25,0%
UNSUPPORTED	18,0%	31,0%
DATE_TIME	11,5%	21,5%
ENTITY	15,5%	12,0%

Nhận xét. Mỗi tỷ lệ trong Bảng 14 được tính trên 200 summary của từng hệ thống. Scratch Transformer gặp lỗi RELATION nhiều hơn ViT5, với $61/200 = 30,5\%$ so với $37/200 = 18,5\%$, cho thấy mô hình còn gặp khó khăn khi liên kết chủ thể, hành động và sự kiện. Ngược lại, ViT5 có tỷ lệ UNSUPPORTED ($62/200 = 31,0\%$) và DATE_TIME ($43/200 = 21,5\%$) cao hơn Scratch Transformer, cho thấy mô hình thường bổ sung thông tin không được source hỗ trợ hoặc xử lý chưa chính xác các mốc thời gian.

Lỗi PREDICATE gần như tương đương giữa Scratch Transformer và ViT5, lần lượt là $49/200 = 24,5\%$ và $50/200 = 25,0\%$, trong khi lỗi ENTITY của Scratch Transformer cao hơn nhẹ. Kết quả gợi ý Scratch Transformer cần ưu tiên cải thiện khả năng biểu diễn quan hệ, còn ViT5 cần tăng cường kiểm soát thông tin không có căn cứ, đặc biệt là ngày tháng và số liệu. Một summary có thể mang nhiều nhãn lỗi, vì vậy tổng các tỷ lệ có thể vượt quá 100%.

VII. THẢO LUẬN

A. Vì sao ROUGE và LLM judge xếp hạng khác nhau?

ROUGE thường mức độ trùng với reference. Scratch Transformer học được phong cách và nội dung gần reference nên đạt điểm cao. LLM judge đọc source và đánh giá mức độ hỗ trợ của từng khẳng định. Lead baselines trích nguyên văn nên gần như không tạo hallucination, trong khi reference có thể chứa chi tiết không được source hỗ trợ. Vì vậy, Lead-1/Lead-3 có thể thấp hơn theo ROUGE nhưng cao hơn rõ rệt về faithfulness.

B. Scratch Transformer so với ViT5

Scratch Transformer đạt ROUGE cao hơn ViT5 trên tập test 2.000 mẫu và có tỷ lệ major factual error thấp hơn ViT5 trong mẫu judge. ViT5 có fluency và conciseness cao hơn. Overall của hai hệ thống sinh chưa tách biệt rõ. Nếu ưu tiên bám reference và kiểm soát mô hình tự xây dựng, Scratch Transformer là nền tảng hợp lý. Nếu ưu tiên độ trôi chảy và súc tích, ViT5 có lợi thế. Tuy nhiên, factuality vẫn là nút thắt của cả hai.

C. Kết luận gắn với mục tiêu sử dụng

Không có một “quán quân” duy nhất: Scratch Transformer tốt nhất theo ROUGE; Lead-3 tốt nhất theo rubric LLM hiện tại nhưng dài hơn rõ rệt; ViT5 trôi chảy và súc tích hơn Scratch; còn khác biệt overall giữa hai mô hình sinh chưa có ý nghĩa rõ. Lựa chọn hệ thống phải gắn với yêu cầu triển khai và chi phí của lỗi factual.

Bảng 15 tổng hợp trách nhiệm chính và sản phẩm kỹ thuật của từng thành viên, được đặt tại ranh giới giữa phần thảo luận và phần mô tả tổ chức thực nghiệm.

TABLE XV. Phân công và đóng góp kỹ thuật của các thành viên.

Thành viên	Trách nhiệm chính	Đóng góp cụ thể
Trần Đức Thịnh	Huấn luyện Scratch Transformer	Xây dựng và vận hành quy trình huấn luyện Transformer encoder-decoder từ đầu; theo dõi train loss và validation loss; lựa chọn checkpoint tốt nhất; chuẩn hóa cấu hình huấn luyện và các siêu tham số phục vụ tái lập.
Lưu Xuân Tân	Đối sánh với ViT5	Thiết lập pipeline suy luận cho ViT5; chuẩn hóa đầu vào, đầu ra và cấu hình giải mã; thực hiện so sánh ViT5 với Scratch Transformer và các baseline trên cùng tập dữ liệu, cùng evaluator và cùng chỉ số.
Đặng Minh Nhật	Đánh giá ROUGE và LLM-as-a-Judge	Thực hiện đánh giá ROUGE và compression ratio trên tập test; thiết kế và triển khai giao thức LLM-as-a-Judge theo hướng source-grounded, làm mù tên hệ thống và cân bằng vị trí candidate; tổng hợp điểm, lỗi factual và phân tích ghép cặp.
Đặng Duy Anh	Thực nghiệm validation và lựa chọn cấu hình	Xây dựng tập cấu hình validation; chạy và tổng hợp các biến thể greedy, beam search, length penalty và no-repeat n-gram; đề xuất cấu hình cuối cho Scratch Transformer và ViT5 trước khi khóa cấu hình để đánh giá test.

VIII. PHÂN CÔNG VÀ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN

Công việc được phân chia theo các khối kỹ thuật chính của pipeline nhằm bảo đảm mỗi thành phần có người phụ trách rõ ràng, đồng thời các kết quả cuối được kiểm tra chéo trong toàn nhóm.

Nhận xét về tổ chức thực nghiệm. Cách phân công trên ánh xạ trực tiếp vào bốn giai đoạn của nghiên cứu: huấn luyện mô hình, xây dựng mô hình đối sánh, lựa chọn cấu hình và đánh giá cuối. Việc tách validation khỏi test giúp phần lựa chọn cấu hình không bị ảnh hưởng bởi kết quả cuối; việc tách đánh giá ROUGE khỏi LLM-as-a-Judge giúp hạn chế kết luận dựa trên một thước đo duy nhất. Dù mỗi thành viên có trách nhiệm chính riêng, các manifest dữ liệu, cấu hình chạy và bảng kết quả được dùng chung để bảo đảm khả năng kiểm tra chéo và tính nhất quán của toàn bộ báo cáo.

IX. HẠN CHẾ VÀ KHẢ NĂNG TÁI LẬP

Candidate evaluation mới sử dụng một LLM judge duy nhất, GPT-5.6 Solar Max, truy cập ngày 22/08/2026; blinding và cân bằng vị trí không loại bỏ mọi thiên kiến theo phong cách, độ dài, xu hướng ưu tiên văn bản trích xuất hoặc đặc tính riêng của judge. Mẫu 200 chỉ bằng 10% test core. Bootstrap mô tả bất định lấy mẫu nhưng không khắc phục sai lệch hệ thống của judge. Chưa có human evaluation độc lập quy mô lớn. Reference không đồng đều có thể làm ROUGE đánh giá thấp prediction đúng source nhưng không trùng chi tiết ngoài source. Ngoài ra, lead bias của tin tức khiến kết quả Lead-1/Lead-3 khó chuyển trực tiếp sang thể loại khác.

X. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Nghiên cứu xây dựng pipeline đánh giá tóm tắt tiếng Việt theo nhiều chiều. Scratch Transformer cạnh tranh tốt theo ROUGE, nhưng đánh giá source-grounded cho thấy các mô hình sinh vẫn gặp vấn đề factuality đáng kể. Lead-3 có coverage và faithfulness cao nhưng đầu ra dài; ViT5 có lợi thế về fluency và conciseness. Reference audit giải thích vì sao không nên đồng nhất ROUGE với chất lượng tuyệt đối.

Hướng phát triển ưu tiên là cải thiện factuality mà không làm giảm coverage: fine-tuning với negative samples, contrastive objectives, sinh nhiều candidate rồi rerank bằng factuality scorer, kiểm chứng thực thể và số liệu sau sinh, và đánh giá lại trên tập con được audit kỹ. Một judge thứ hai hoặc human audit tập trung vào lỗi nghiêm trọng sẽ giúp kiểm tra độ bền của kết luận.

REFERENCES

- [1] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [2] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, 2004, pp. 74–81.
- [3] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text Transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [4] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” in *Proc. EMNLP: System Demonstrations*, 2018, pp. 66–71.
- [5] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.
- [7] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” arXiv:1606.08415, 2016.
- [8] L. Phan, H. Tran, H. Nguyen, and T. H. Trinh, “ViT5: Pretrained text-to-text Transformer for Vietnamese language generation,” in *Proc. NAACL 2022 Student Research Workshop*, 2022, pp. 136–142, doi: 10.18653/v1/2022.naacl-srw.18.